ObjectIR

Version 2

Language & Runtime Specification

A stack-based intermediate representation and virtual machine
for the .NET / Unity ecosystem

Namespace:	`ObjectIR.Core`
IR format:	TextIR, JSON (`ModuleDto`), FOB/IR v3 binary
Target:	.NET 10 / Unity 2022+
Revision:	March 21, 2026

This document describes the ObjectIR instruction set, TextIR source format, module representation, runtime API, and CLR interoperability layer.

Contents

1 Overview **3**
1.1 Design Goals . . . 3
1.2 Quick-start . . . 3

2 Module Structure **3**

3 Input Formats **4**

4 TextIR Syntax **4**
4.1 Module Header . . . 4
4.2 Type Declarations . . . 5
4.3 Member Declarations . . . 5
4.4 Annotations / Attributes . . . 5
4.5 Inline String Formatting . . . 6

5 Type System **6**
5.1 Primitive Types . . . 6
5.2 User-Defined Types . . . 6
5.3 CLR-Backed Types . . . 7

6 Instruction Set Reference **7**
6.1 Stack Manipulation . . . 7
6.2 Load Constants . . . 7
6.3 Arguments and Local Variables . . . 7
6.4 Field Access . . . 7
6.5 Arithmetic . . . 8
6.6 Logical . . . 8
6.7 Comparison . . . 8
6.8 Type Conversion and Testing . . . 9
6.9 Object and Array Creation . . . 9
6.10 Method Calls . . . 9
6.10.1 Call Resolution Order . . . 9
6.11 Control Flow . . . 10
6.11.1 Condition System . . . 10

7 FOB/IR v3 Binary Format **10**
7.1 Binary Format Summary . . . 10
7.2 Binary Opcode Map . . . 11

8 Runtime API **11**
8.1 Constructor . . . 11
8.2 Module Loading . . . 12
8.3 Execution . . . 12
8.4 Module Inspection . . . 12
8.5 Static Field Access . . . 12
8.6 Observability Hooks . . . 13
8.7 Public Properties . . . 13

9 CLR Type Registration **13**
9.1 Example . . . 13

10 Native Method Registration **14**

10.1 Delegate Signature . . . 14
10.2 Registration . . . 14
10.3 Built-in Native Methods . . . 14

11 Value Coercion **15**

12 Exception Model **15**
12.1 Raising Exceptions . . . 15
12.2 Structured Handling . . . 15
12.3 Control-Flow Sentinels . . . 15
12.4 Exception-to-CLR Conversion . . . 16

13 Attributes **16**

14 Data-Transfer Object Types **16**
14.1 `ModuleDto` . . . 16
14.2 `TypeDto` . . . 16
14.3 `MethodDto` . . . 17
14.4 `InstructionDto` . . . 17
14.5 `CallTargetDto` . . . 17
14.6 `ConditionDto` . . . 17
14.7 Supporting DTOs . . . 17

15 Runtime Support Types **18**
15.1 `CallFrame` . . . 18
15.2 `ManagedObject` . . . 18
15.3 `ValueHelpers` . . . 18

16 Thread Safety **18**

17 Worked Examples **19**
17.1 Hello World . . . 19
17.2 Fibonacci . . . 19
17.3 Instance Classes and Fields . . . 20

18 Limitations and Known Gaps **20**

Overview

ObjectIR is a stack-based intermediate representation (IR) and accompanying virtual machine written in C# (.NET 10). A program is described as a collection of typed modules, each containing class and interface definitions with methods composed of explicit stack-machine instructions.

The runtime loads a module, resolves types and methods, and executes instructions against a per-call-frame evaluation stack. Every value on the stack is an untyped `object?`; conversions are performed lazily by the `ValueHelpers` layer.

Design Goals

- **Human-readable source.** The canonical representation is *TextIR*: a whitespace-significant, keyword-driven text format that compiles to the same in-memory `ModuleDto` as the JSON and binary forms.
- **Multiple input formats.** A module may be loaded from TextIR source, a serialised JSON `ModuleDto`, a pre-parsed `ModuleNode` AST, or a compiled FOB/IR v3 binary.
- **Embeddable.** Host applications load a module, call IR methods, and register CLR types or native callbacks without touching a parser.
- **Reflection fallback.** Any CLR type that does not have a hand-written binding is handled automatically through reflection, so even obscure APIs work without extra glue code.
- **Struct boxing.** Value types such as `Vector3` and `Quaternion` are transparently wrapped in `ManagedObject` instances so the IR heap can hold them without loss of precision.

Quick-start

Listing 1: Minimal embedding example

```
// Construct a runtime (Lattice standalone or Unity variant).
var rt = new IRRuntime();

// Load a module in any supported format.
rt.LoadModule(textIrSource);          // auto-detects TextIR / JSON
rt.LoadModule(fobBytes);              // raw FOB/IR v3 binary
rt.LoadFobFile("assets/game.fob");    // from disk

// Run the canonical entry point: static Program.Main()
rt.Run();

// Invoke individual methods by qualified name.
rt.CallMethod("Player.OnUpdate", deltaTime);
float hp = rt.CallMethod<float>("Player.GetHealth");

// Inspect module metadata without executing code.
if (rt.HasMethod("Program.OnFixedUpdate"))
    rt.CallMethod("Program.OnFixedUpdate");
```

Module Structure

An ObjectIR *module* is the top-level compilation unit. It contains:

- A **name** and a three-part **version** number (`major.minor.patch`).
- An ordered list of **type definitions** (`TypeDto`): classes, interfaces, structs, and enums.

- A (currently reserved) **functions** table for future module-level free functions.

Modules are represented in memory as a `ModuleDto` (see Section 14). The runtime holds exactly one active module at a time; loading a new module replaces the previous one.

Input Formats

The runtime accepts four equivalent input representations selected via the `InputFormat` enum or auto-detected.

Format	Enum member	Description
`Auto`	`InputFormat.Auto`	Inspects the first character of the input string. `{` or `[` triggers JSON parsing; everything else is treated as TextIR source.
`TextIr`	`InputFormat.TextIr`	Human-readable TextIR source (see Section 4). Parsed by `TextIrParser` then lowered to `ModuleDto` by `IrAstLowering`.
`Json`	`InputFormat.Json`	A serialised `ModuleDto` JSON object decoded with `System.Text.Json.JsonSerializer` (camelCase property names).
`Ast`	`InputFormat.Ast`	Alias for `TextIr`; the string is processed through the same parser / lowering pipeline.
`FobIr`	`InputFormat.FobIr`	The string argument is treated as a *file path* to a FOB/IR v3 binary. The file is read with `File.ReadAllBytes` and decoded by `FobIrReader` + `IrModuleBinaryReader` (see Section 7).

Note: `LoadModule(byte[])` and `LoadFobFile(string)` always decode the input as FOB/IR v3 binary, bypassing the `InputFormat` enum entirely.

TextIR Syntax

TextIR is the canonical human-readable source format for ObjectIR modules. A TextIR file begins with a module header followed by one or more type definitions.

Listing 2: Minimal TextIR module

```
module MyApp version 1.0.0

class Program {
    static method Main() -> void {
        ldstr "Hello, ObjectIR!"
        call System.Console.WriteLine(string) -> void
        ret
    }
}
```

Module Header

```
module <name> version <major>.<minor>[.<patch>]
```

Type Declarations

Keyword	Meaning
`class`	Reference type (heap-allocated `ManagedObject`)
`interface`	Interface (pure virtual contract)
`struct`	Value type
`enum`	Enumeration type

Access modifiers (`public`, `private`, `protected`, `internal`) and type modifiers (`abstract`, `sealed`) may precede the keyword. Inheritance and interface implementation use `::`

```
class Animal {
    field name: string
    virtual method Speak() -> void { ret }
}

class Dog : Animal {
    override method Speak() -> void implements Animal.Speak {
        ldstr "Woof!"
        call System.Console.WriteLine(string) -> void
        ret
    }
}
```

Member Declarations

```
// Field declaration
[static] field <name>: <type>

// Method declaration
[static] [virtual] [override] [abstract] method <name>(<params>) -> <rettype>
    [implements <Interface>.<MethodName>]
{
    [local <name>: <type>]*
    <instructions>
}

// Constructor
constructor(<params>) {
    [local <name>: <type>]*
    <instructions>
}
```

Parameter and local variable lists share the same syntax: `name: type[, name: type, ...]`

Annotations / Attributes

Custom attributes may be applied to types, fields, and methods using the `@` prefix immediately before the declaration:

```
@Serializable
class PlayerData {
    @NonSerialized
    field cachedHash: int32

    @Deprecated("Use SaveAsync instead")
```

```
    method Save() -> void { ret }
}
```

At runtime the attribute is instantiated as a `ManagedObject` and attached to the corresponding IR object.

Inline String Formatting

Within `ldstr` operands, the following C-style escape sequences are recognised: `\n`, `\t`, `\r`, `\\`, `\"`.

Type System

Primitive Types

Type names in IR operands are **case-insensitive**. The runtime normalises them to lower-case before lookup, so `System.String`, `system.string`, and `string` are all equivalent.

IR name(s)	Alias(es)	CLR type
`void`	`system.void`	`void`
`bool`	`system.boolean`	`bool`
`int8`	`system.sbyte`	`sbyte`
`uint8`	`system.byte`	`byte`
`int16`	`system.int16`	`short`
`uint16`	`system.uint16`	`ushort`
`int32`	`system.int32`	`int`
`uint32`	`system.uint32`	`uint`
`int64`	`system.int64`	`long`
`uint64`	`system.uint64`	`ulong`
`float32`, `single`	`system.single`	`float`
`float64`, `double`	`system.double`	`double`
`char`	`system.char`	`char`
`string`	`system.string`	`string`
`object`	`system.object`	`object`
`decimal`	`system.decimal`	`decimal`
`datetime`	`system.datetime`	`DateTime`
`timespan`	`system.timespan`	`TimeSpan`
`guid`	`system.guid`	`Guid`

User-Defined Types

User-defined types declared in the module are managed by the VM as `ManagedObject` instances at runtime. Each instance carries:

- A **type name string** (`TypeName`).
- A **field dictionary** (`Dictionary<string, object?>`) for named instance fields.
- An optional **CLR backing object** (see Section 5.3).
- Attached **attribute instances** (see Section 13).
- Arbitrary **metadata slots**.

CLR-Backed Types

Any CLR type registered via `RegisterClrType` is treated as a first-class IR type. When `newobj` creates an instance the runtime calls `Activator.CreateInstance` and stores the result as the `ClrInstance` of the wrapping `ManagedObject`. Field and method access falls through to reflection on the real CLR object.

Instruction Set Reference

The execution engine is a **stack machine**. The call stack is a `Stack<CallFrame>`; each frame owns:

- An evaluation stack (`Stack<object?>`).
- A named argument table and local variable table.
- A pending-exception slot.
- A program counter (instruction index within the method body).

All operands are stored as raw `JsonElement` values and decoded lazily by the instruction handler.

Stack Manipulation

Opcode	Description
`nop`	No-operation.
`dup`	Duplicate the top value on the evaluation stack.
`pop`	Discard the top value.
`ldnull`	Push `null` onto the stack.

Load Constants

Opcode	Description
`ldc`	Push a typed constant. Operand: `{value, type}`. `type` defaults to `int64`.
`ldc.i4`	Push a 32-bit integer constant.
`ldc.i8`	Push a 64-bit integer constant.
`ldc.r4`	Push a 32-bit float constant.
`ldc.r8`	Push a 64-bit float constant (double).
`ldstr`	Push a string constant. Operand: `{value}` or bare string.

Arguments and Local Variables

Opcode	Description
`ldarg`	Push a method argument by index or name. Index `0` (or name `this`) is the receiver for instance methods.
`starg`	Pop a value and store it in a named argument slot.
`ldloc`	Push a local variable by name.
`stloc`	Pop a value and store it in a named local variable.

Field Access

Opcode	Description
`ldfld`	Pop an object from the stack (or use `this`) and push the named instance field. If the object has a CLR backing instance, reflection is used to read the corresponding property/field.
`stfld`	Pop a value, then pop an object; store the value in the named instance field. Writes through to the CLR backing instance when present.
`ldsfld`	Push a static field. Operand: `{field: {declaringType, name}}`.
`stsfld`	Pop a value and store it in a static field.

Arithmetic

Opcode	Description
`add`	Pop b, pop a, push $a + b$.
`sub`	Pop b, pop a, push $a - b$.
`mul`	Pop b, pop a, push $a \times b$.
`div`	Pop b, pop a, push $a \div b$.
`rem`	Pop b, pop a, push $a \bmod b$.
`neg`	Pop v, push $-v$.

If either operand is a `float`, `double`, or `string`, both operands are promoted to `double` mode; otherwise integer (`int64`) mode is used. String concatenation applies for **`add`** when either operand is a `string`.

Logical

Opcode	Description
`not`	Pop v, push `!ToBool(`v`)`.

Comparison

Each opcode pops b then a and pushes a `bool`.

Opcode	Condition
`ceq`	$a = b$
`cne`	$a \neq b$
`cgt`	$a > b$
`cge`	$a \geq b$
`clt`	$a < b$
`cle`	$a \leq b$

String comparisons use `StringComparison.Ordinal`. Boolean operands are supported for **`ceq`**/**`cne`**. All other operands are promoted to `double` before comparison.

Type Conversion and Testing

Opcode	Description
`conv`	Pop a value and push it converted to the named target type. Uses `ValueHelpers.ConvertTo`.
`castclass`	Assert that the top-of-stack `ManagedObject` has the expected type name; throws `InvalidCastException` if it does not.
`isinst`	Pop a value; push `true` if it is a `ManagedObject` with the given type name, otherwise `false`.

Limitation: `castclass` and `isinst` perform *exact* type name matching only. Inheritance-aware checks for IR-defined types are not supported in the current version.

Object and Array Creation

Opcode	Description
`newobj`	Create a new `ManagedObject` of the named type. If the type is registered as a CLR type, a real CLR instance is created via `Activator.CreateInstance` and attached to the wrapper.
`newarr`	Push a new `List<object?>` (dynamically-typed, variable-length array).
`ldelem`	Pop an index (0-based), pop an array, push `array[index]`.
`stelem`	Pop a value, pop an index, pop an array; set `array[index] = value`.

Arrays in ObjectIR v2 are implemented as `List<object?>`. Typed arrays (e.g. `int[]` or `float[]`) are not distinguished at runtime.

Method Calls

Opcode	Description
`call`	Static or instance call. Pops arguments in declaration order (left-to-right), then pops the instance for non-static targets. Operand: `{method: {declaringType, name, returnType, parameterTypes[]}}`.
`callvirt`	Virtual call. Identical to `call` but additionally dispatches through the virtual method table if the target is an IR-defined `ManagedObject`.
`ret`	Return from the current frame. If a value is present on the evaluation stack it is transferred to the caller's stack.

Call Resolution Order

Both `call` and `callvirt` apply the following dispatch sequence on every invocation:

1. **Exact native lookup.** Full signature match in the `_nativeMethods` dictionary (e.g. `System.Console.WriteLin`
2. **Simplified native lookup.** Parameter types reduced to unqualified, lower-case short names (e.g. `System.String` → `string`) and re-tried.

3. **Reflection dispatch.** When `EnableReflectionNativeMethods` is `true`, the CLR type registry is consulted and the method is invoked via reflection on the backing CLR instance.
4. **IR method dispatch.** Lookup in the loaded module's method table.

Control Flow

Opcode	Description
`if`	Evaluate a condition (see Section 6.11.1); if truthy execute `thenBlock`, otherwise execute the optional `elseBlock`.
`while`	Repeatedly evaluate a condition and execute `body`. Supports **`break`** and **`continue`**.
`break`	Exit the nearest enclosing **`while`** loop. Implemented via an internal `BreakSignal` exception.
`continue`	Skip to the next iteration of the nearest **`while`** loop. Implemented via `ContinueSignal`.
`try`	Structured exception handling with `tryBlock`, zero or more `catchBlocks`, and an optional `finallyBlock`.
`throw`	Pop a value and store it as the pending exception on the current frame.

Condition System

Conditions embedded in **`if`** or **`while`** operands are `ConditionDto` objects evaluated by the `EvaluateCondition` helper:

Kind	Evaluation
`stack`	Pop the top-of-stack value and coerce it to `bool`. An empty stack is treated as `false`.
`binary`	Pop r, pop l; compare using the specified `operation` opcode (**`ceq`**, **`cgt`**, etc.).
`expression`	Execute a single embedded `InstructionDto` and pop the result.
`block`	Execute a sequence of `InstructionDto`s and pop the final result.

FOB/IR v3 Binary Format

The FOB/IR v3 binary format provides a compact, pre-parsed representation of an ObjectIR module. Loading a binary module is a two-stage process:

1. `FobIrReader` strips the FOB envelope (magic bytes, header, compression metadata).
2. `IrModuleBinaryReader` decodes the payload into a `ModuleDto` equivalent.

Binary Format Summary

Section	Contents
Header	Magic number, format version, module name, version triple.

Section	Contents
Type table	Sequential `TypeDto` records (kind, name, namespace, fields, methods, base type, interfaces).
Method body	Per-method: parameter list, local variable list, instruction count, instruction records.
String pool	Interned string table referenced by index throughout the binary.

Binary Opcode Map

Each instruction is serialised as a 1-byte opcode followed by a variable-length operand. The byte values mirror the `OpCodes` enum defined in `ObjectIR.Core`.

Byte	Mnemonic	Byte	Mnemonic
0x00	nop	0x10	ldsfld
0x01	ldc	0x11	stsfld
0x02	ldstr	0x12	newobj
0x03	ldarg	0x13	newarr
0x04	starg	0x14	ldelem
0x05	ldloc	0x15	stelem
0x06	stloc	0x16	call
0x07	add	0x17	callvirt
0x08	sub	0x18	ret
0x09	mul	0x19	if
0x0A	div	0x1A	while
0x0B	rem	0x1B	break
0x0C	neg	0x1C	continue
0x0D	ceq	0x1D	try
0x0E	cne	0x1E	throw
0x0F	ldfld	0x1F	conv
0x20	castclass	0x21	isinst
0x22	dup	0x23	pop
0x24	ldnull	0x25	not
0x26	cgt	0x27	cge
0x28	clt	0x29	cle
0x2A	stfld	0x2B	ldc.i4
0x2C	ldc.i8	0x2D	ldc.r4
0x2E	ldc.r8		

Runtime API

Constructor

```
// Standalone (Lattice) runtime:
public IRRuntime(string? irSource = null,
                 bool enableReflectionNativeMethods = false)

// Unity-aware runtime:
public UnityIRRuntime(GameObject? owner = null)
```

On first instantiation (per process) both constructors trigger a one-time, thread-safe registration of all built-in CLR types (`EnsureBuiltinTypesRegistered`) followed by setup of the pre-registered native method bindings (`RegisterManualBindings`).

Module Loading

```
// Load from string (auto-detects TextIR / JSON)
public void LoadModule(string input, InputFormat format = InputFormat.
    Auto)

// Load from a pre-parsed ModuleNode AST
public void LoadModule(ModuleNode astModule)

// Load from raw FOB/IR v3 binary bytes
public void LoadModule(byte[] fobBytes)

// Convenience: read a FOB file from disk
public void LoadFobFile(string path)

// Full reset: clears call stack and static fields, then loads
public void LoadProgram(string input, InputFormat format = InputFormat.
    Auto)
```

Execution

```
// Execute static Program.Main() (no parameters)
public void Run()

// Invoke any method by qualified name
public object? CallMethod(string qualifiedName, params object?[] args)

// Generic variant with automatic return-value conversion
public TResult CallMethod<TResult>(string qualifiedName, params object?[]
     args)
```

`qualifiedName` follows the format `"TypeName.MethodName"` or just `"MethodName"` for unqualified lookup. Throws `InvalidOperationException` when no module is loaded or the type cannot be found; throws `MissingMethodException` when the method is absent.

Module Inspection

```
// Returns true if the loaded module contains the specified method
public bool HasMethod(string qualifiedName)
```

This performs IR metadata inspection only — no code is executed.

Static Field Access

```
// Two-argument form
public object? GetStaticField(string declaringType, string fieldName)

// "TypeName.FieldName" single-string form
public object? GetStaticField(string qualifiedName)
```

Observability Hooks

```
// Invoked before every instruction (debugger / profiler hook)
public Action<CallFrame, InstructionDto>? OnStep { get; set; }

// Invoked when an unhandled exception escapes the execution loop
public Action<Exception>? OnException { get; set; }
```

Public Properties

Property	Type	Description
`CallStack`	`IReadOnlyList<CallFrame>`	Read-only view of the call stack (outermost first).
`StaticFields`	`IReadOnlyDictionary`	All static field values written by `stsfld`, keyed by `(declaringType, fieldName)`.
`EnableReflectionNativeMethods`	`bool`	When `true`, unresolved calls are retried via CLR reflection before consulting the IR method table.

CLR Type Registration

Host code registers CLR types so the VM can instantiate them, access their fields, and invoke their methods as if they were IR-defined.

```
// Register by type parameter (CLR FullName used as IR name by default)
IRRuntime.RegisterClrType<MyService>();

// Register with a custom IR name
IRRuntime.RegisterClrType<MyService>("Services.MyService");

// Register via a Type object
IRRuntime.RegisterClrType("Services.MyService", typeof(MyService));
```

The registration is **process-global** (stored in a static, thread-safe dictionary). Once registered:

- **`newobj`** creates a real CLR instance via `Activator.CreateInstance` and attaches it to the `ManagedObject` wrapper.
- **`ldfld`** / **`stfld`** use reflection to read/write properties and fields on the CLR instance, falling back to the VM field dictionary if the member is not found.
- **`ldsfld`** / **`stsfld`** reflectively access static CLR fields and properties.
- Reflected **`call`**/**`callvirt`** receive the real CLR instance, not the `ManagedObject` wrapper.

Example

```
public class Counter {
    public int Value { get; set; }
    public void Increment() => Value++;
    public int GetValue() => Value;
}

IRRuntime.RegisterClrType<Counter>("Counter");
```

```
var rt = new IRRuntime("""
    module CounterDemo version 1.0.0

    class Program {
        static method Main() -> void {
            newobj Counter
            callvirt Counter.Increment() -> void
            callvirt Counter.GetValue() -> int32
            call System.Console.WriteLine(int32) -> void
            ret
        }
    }
""", enableReflectionNativeMethods: true);

rt.Run(); // prints: 1
```

Native Method Registration

Delegate Signature

```
public delegate object? NativeMethod(object? instance, object?[] args);
```

`instance` is `null` for static calls and a `ManagedObject` (wrapping `this`) for instance/virtual calls. `args` contains positional arguments popped from the evaluation stack in declaration order.

Registration

```
public void RegisterNativeMethod(string signature, NativeMethod method)
```

Adds or replaces a native binding in the instance-level `_nativeMethods` dictionary. The `signature` format is:

`DeclaringType.MethodName(param1Type,param2Type,...)`

Both the fully-qualified form (e.g. `System.String`) and the short lower-case form (e.g. `string`) are tried automatically during dispatch, so registering either form covers both.

Built-in Native Methods

The following methods are pre-registered on every runtime instance.

Signature	Behaviour
`System.Console.WriteLine(string)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(int32)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(object)`	`Console.WriteLine(args[0])`
`System.Console.WriteLine(string,string)`	Concatenates args and writes a line
`System.Console.WriteLine(string,object)`	`string.Format` then writes a line
`System.Console.Write(string)`	`Console.Write(args[0])`
`System.Console.Write(string,string)`	Concatenates args and writes
`System.Console.ReadLine()`	Returns `Console.ReadLine()`
`System.Console.Clear()`	`Console.Clear()`
`System.Console.Beep()`	`Console.Beep()`
`System.String.Concat(string,string)`	`string.Concat(args[0], args[1])`

Signature	Behaviour
`System.String.Format(string,object)`	`string.Format(fmt, args[1])`
`System.String.Format(string,object,object)`	`string.Format(fmt, a, b)`
`System.String.IsNullOrEmpty(string)`	`string.IsNullOrEmpty(args[0])`
`System.Object.ToString()`	`instance?.ToString()`
`System.Object.GetType()`	`instance?.GetType()`

Value Coercion

`ValueHelpers` provides stateless type coercion used throughout the runtime. The `ToBool`, `ToLong`, `ToDouble`, and `ConvertTo` methods apply the following rules:

Target type	Conversion rule
`bool`	`null`, `0`, and `""` convert to `false`; all other values convert to `true`. Numeric-string `"0"` also converts to `false`.
`long`	Numeric types are truncated/cast. Strings are parsed with `CultureInfo.InvariantCulture`.
`double`	Same as `long` but without truncation.
`string`	`value.ToString()`, or `null` if `value` is `null`.
CLR value type	Typed fast-paths (`int`, `float`, etc.) then `Convert.ChangeType` fallback.

Exception Model

Raising Exceptions

The **`throw`** instruction pops any value from the evaluation stack (a `ManagedObject`, a CLR `Exception`, or a plain `string`) and stores it as `frame.PendingException`.

Structured Handling

The **`try`** instruction provides first-class structured exception handling with `tryBlock`, zero or more `catchBlocks`, and an optional `finallyBlock`. Catch clauses match in declaration order:

1. **Catch-all** (no type specified) — matches anything.
2. **IR type name match** — the pending exception is a `ManagedObject` whose `TypeName` equals the catch type.
3. **CLR type match** — `Type.IsAssignableFrom` check against the CLR type registered for the catch-type name.

If no catch clause matches, the pending exception propagates up the call stack. At the top of the stack it is delivered to the `OnException` callback.

Control-Flow Sentinels

- `BreakSignal` — lightweight internal `Exception` used to implement **`break`**. Caught exclusively inside the **`while`** instruction handler; never visible to user code.
- `ContinueSignal` — same mechanism for **`continue`**.
- `ExceptionSignal` — carries an unhandled IR exception object out of the execution loop to the host when no catch block matches.

Exception-to-CLR Conversion

When an IR `ManagedObject` exception escapes unhandled, the runtime constructs a CLR `Exception` with:

- The IR type name and `message` field (if present) as the exception message.
- A source location extracted from the instruction operand (`location`, `loc`, or `sourceLocation` JSON properties; line number from `line`, `startLine`, or `start`).

Attributes

Custom attributes can be applied to IR types, fields, and methods using the `@AttributeName(args)` syntax in TextIR. At runtime:

1. The runtime instantiates each attribute as a `ManagedObject` via `AttachAttributesFromIR`.
2. Attribute instances are stored in a list on the target `ManagedObject`.
3. Host code retrieves them with:

```
ManagedObject instance = ...;
var attr = instance.GetAttribute<MyManagedAttribute>();
```

Data-Transfer Object Types

These types form the in-memory representation of a compiled IR module and are shared across all runtime variants (Lattice standalone, UnityIRRuntime, etc.).

`ModuleDto`

```
public sealed class ModuleDto
{
    public string      name       { get; set; }
    public string      version    { get; set; }
    public TypeDto[]   types      { get; set; }
    public JsonElement functions  { get; set; }  // reserved
}
```

Top-level container. `types` holds all class/interface definitions. `functions` is reserved for future module-level free functions.

`TypeDto`

```
public sealed class TypeDto
{
    public string     kind;        // "class" | "interface" | "struct" |
        "enum"
    public string     name;
    public string?    _namespace;
    public string?    baseType;
    public FieldDto[]  fields;
    public MethodDto[] methods;
    public JsonElement baseInterfaces, genericParameters, properties;
}
```

MethodDto

```
public sealed class MethodDto
{
    public string            name, returnType;
    public bool              isStatic, isVirtual, isOverride, isAbstract;
    public ParameterDto[]    parameters;
    public LocalVariableDto[] locals;
    public InstructionDto[]  instructions;
    public AttributeDto[]    attributes;
}
```

InstructionDto

```
public sealed class InstructionDto
{
    public string      opCode  { get; set; }
    public JsonElement operand { get; set; }
}
```

The operand is kept as a raw `JsonElement` so the VM can lazily decode only the fields required by each opcode.

CallTargetDto

```
public sealed class CallTargetDto
{
    public string   declaringType;
    public string   name;
    public string   returnType;
    public string[] parameterTypes;
}
```

The lookup key is assembled as:

```
declaringType.name(parameterTypes[0],parameterTypes[1],...)
```

ConditionDto

```
public sealed class ConditionDto
{
    public string           kind;        // "stack"|"binary"|"expression
        "|"block"
    public string?          operation;   // for "binary" kind
    public InstructionDto[] body;        // for "block" kind
    public ConditionKind    Kind { get; }  // computed enum
}
```

Supporting DTOs

Type	Purpose
`FieldDto`	Field declaration (name, type, static flag, attributes).
`ParameterDto`	Method parameter (name, type).
`LocalVariableDto`	Local variable declaration (name, type).

Type	Purpose
`AttributeDto`	Custom attribute application (name, positional args).
`FieldRefDto`	Reference to a field (`declaringType`, `name`).

Runtime Support Types

`CallFrame`

Represents a single activation record on the call stack.

Member	Description
`Method`	The `MethodDto` being executed.
`Stack`	The per-frame evaluation stack (`Stack<object?>`).
`Locals`	Named local variable dictionary.
`Args`	Named argument dictionary (includes `this`).
`IP`	Current instruction index (program counter).
`PendingException`	Exception object awaiting a catch clause, or `null`.
`ReturnValue`	Value returned by a nested call, or `null`.

`ManagedObject`

Represents any heap-allocated value in the IR runtime (classes, value-type wrappers, exception objects, . . .).

Member	Description
`TypeName`	IR type name string.
`Fields`	`Dictionary<string, object?>` of instance field values.
`ClrInstance`	Optional CLR backing object (`object?`).
`Attributes`	List of attached `AttributeDto` instances.
`Metadata`	Arbitrary key/value metadata dictionary.

`ValueHelpers`

A stateless, `internal static` helper class.

Method	Description
`ToBool(object?)`	Coerce any value to `bool` (see Section 11).
`ToLong(object?)`	Coerce to `long`.
`ToDouble(object?)`	Coerce to `double`.
`ConvertTo(object?, Type)`	General-purpose type conversion with fast-path table.
`ArithmeticMode(a, b)`	Returns `Float` or `Int` based on operand types.

Thread Safety

- The **process-global CLR type registry** (`s_registeredClrTypes`) is protected by a static `ReaderWriterLockSlim` and is safe for concurrent reads and infrequent writes.
- **Individual runtime instances** are not thread-safe. A single `IRRuntime` / `UnityIRRuntime` must not be called from multiple threads simultaneously.

- **One-time initialisation** of built-in type registrations and manual bindings is protected by a static lock and runs at most once per process.

Worked Examples

Hello World

Listing 3: Hello World in TextIR

```
module HelloWorld version 1.0.0

class Program {
    static method Main() -> void {
        ldstr "Hello, World!"
        call System.Console.WriteLine(string) -> void
        ret
    }
}
```

Fibonacci

Listing 4: Recursive Fibonacci

```
module Fibonacci version 1.0.0

class Program {
    static method Fib(n: int32) -> int32 {
        local result: int32

        ldarg n
        ldc { "value": 2, "type": "int32" }
        clt
        if {
            "kind": "stack",
            "thenBlock": [
                { "opCode": "ldarg", "operand": "n" },
                { "opCode": "ret" }
            ]
        }

        ldarg n
        ldc { "value": 1, "type": "int32" }
        sub
        call Program.Fib(int32) -> int32
        ldarg n
        ldc { "value": 2, "type": "int32" }
        sub
        call Program.Fib(int32) -> int32
        add
        ret
    }

    static method Main() -> void {
        ldc { "value": 10, "type": "int32" }
        call Program.Fib(int32) -> int32
        call System.Console.WriteLine(int32) -> void
        ret
    }
}
```

Instance Classes and Fields

Listing 5: Instance class with constructor

```
module ClassDemo version 1.0.0

class Vector2D {
    field x: float32
    field y: float32

    constructor(x: float32, y: float32) {
        ldarg this
        ldarg x
        stfld x
        ldarg this
        ldarg y
        stfld y
        ret
    }

    method Length() -> float32 {
        ldarg this
        ldfld x
        ldarg this
        ldfld x
        mul
        ldarg this
        ldfld y
        ldarg this
        ldfld y
        mul
        add
        call System.Math.Sqrt(float64) -> float64
        conv float32
        ret
    }
}

class Program {
    static method Main() -> void {
        ldc { "value": 3.0, "type": "float32" }
        ldc { "value": 4.0, "type": "float32" }
        newobj Vector2D
        callvirt Vector2D.Length() -> float32
        call System.Console.WriteLine(object) -> void
        ret
    }
}
```

Limitations and Known Gaps

The following features are not yet supported in ObjectIR v2:

- **Generics.** Generic type parameters and generic method instantiation are not supported. All generic APIs must be accessed through native bindings or reflection.
- **Typed arrays.** Arrays are implemented as `List<object?>`; `int[]` and `float[]` are not distinguished at runtime.

- **Exact-match only for `castclass` / `isinst`.** Inheritance chains for IR-defined types are not walked; only exact `TypeName` string equality is checked.
- **Constructor arguments for CLR-backed types.** `newobj` uses the parameterless `Activator.CreateInstan` constructor arguments must be passed via native method wrappers or IR constructors that `stfld` after creation.
- **Module-level free functions.** The `functions` field on `ModuleDto` is reserved and not yet executed by the VM.
- **Concurrency.** Individual runtime instances are single-threaded. Executing IR code from multiple threads on the same instance is not safe.